GROUPE 4

# RAPPORT TECHNIQUE DU GROUPE 4 : Module table2d_moduleTris et Recherches sur Tableaux 2D en Python


***MEMBRES DU GROUPE :***
***Amadou Habib Kane***
***Coumba Gueye***
***Fatou Gueye***
***Manétou Dramé***
***Mouhamed Diop***


Cours : Algorithmique & Structures de Données
Enseignant :PR Mamadou Diahame
Niveau : Licence 3 — GLSI
Année académique : 2025 – 2026

# I. Présentation générale du projet

Ce rapport documente l'analyse et le fonctionnement de deux fichiers Python constituant un projet complet de manipulation de tableaux à deux dimensions (2D). Le projet implémente plusieurs algorithmes classiques de tri et de recherche, appliqués à des structures de données 2D, avec une interface d'utilisation interactive.

## 1.1. Fichiers du projet

| | |
|---|---|
| **Fichier 1** | table2d_module.py — Module principal contenant tous les algorithmes |
| **Fichier 2** | exemple_utilisation.py — Script de démonstration interactive |
| **Langage** | Python 3 |
| **Paradigme** | Programmation modulaire et procédurale |

## 1.2. Objectifs du module

Le module table2d_module.py a trois objectifs principaux :

- Initialiser une table à deux dimensions avec des valeurs aléatoires.
- Trier les éléments d'une table 2D selon plusieurs algorithmes classiques.
- Rechercher un élément dans une table 2D avec différentes méthodes.

# II. Architecture et structure du module

## 2.1. Principe de fonctionnement

Le projet repose sur un principe clé pour simplifier l'application des algorithmes sur des tableaux 2D : toute opération de tri ou de recherche passe par trois étapes :

**Étape 1 — Aplatissement :** La table 2D est convertie en liste 1D.
**Étape 2 — Traitement :** L'algorithme de tri ou de recherche est appliqué sur la liste 1D.
**Étape 3 — Reconstruction :** La liste triée est restructurée en table 2D avec les dimensions d'origine.

Exemple illustré dans la documentation du module :

```
Table 2D initiale : [[7, 2, 9], [1, 5, 3]]
Aplatissement : [7, 2, 9, 1, 5, 3]
Après tri : [1, 2, 3, 5, 7, 9]
Reconstruction : [[1, 2, 3], [5, 7, 9]]
```

## 2.2. Types personnalisés

Le module définit deux alias de type pour améliorer la lisibilité :

| | |
|---|---|
| **Table2D** | List[List[int]] — représente un tableau 2D d'entiers |
| **Coordonnee** | Tuple[int, int] — représente une position (ligne, colonne) |

## 2.3. Organisation des fonctions

Le module est organisé en quatre groupes de fonctions :

| Groupe | Fonctions | Rôle |
|---|---|---|
| Outils généraux | generer_tableau_aleatoire, dimensions_tableau, aplatir_tableau, reconstruire_tableau | Utilitaires de base |
| Tris | tri_bulle, tri_selection, tri_insertion, tri_rapide, tri_fusion | Algorithmes de tri |
| Recherches | recherche_lineaire, recherche_binaire, recherche_dichotomie | Algorithmes de recherche |
| Affichage | afficher_tableau | Visualisation du tableau |

- ### generer_tableau_aleatoire(lignes, colonnes, minimum, maximum)

```python
def generer_tableau_aleatoire(
    lignes: int,
    colonnes: int,
    minimum: int = 0,
    maximum: int = 100,
) -> Table2D:
    """
    Génère une table à deux dimensions initialisée aléatoirement.

    Paramètres :
    - lignes : nombre de lignes du tableau
    - colonnes : nombre de colonnes du tableau
    - minimum : valeur minimale possible
    - maximum : valeur maximale possible

    Retourne :
    - une liste de listes d'entiers

    Exemple :
    >>> generer_tableau_aleatoire(2, 3, 1, 9)
    [
    [4, 8, 1],
    [6, 2, 7]
    ]
    """
    if lignes <= 0 or colonnes <= 0:
        raise ValueError("Le nombre de lignes et de colonnes doit être positif.")
    if minimum > maximum:
        raise ValueError("Le minimum ne peut pas être supérieur au maximum.")
```

```python
    """
    if lignes <= 0 or colonnes <= 0:
        raise ValueError("Le nombre de lignes et de colonnes doit être positif.")
    if minimum > maximum:
        raise ValueError("Le minimum ne peut pas être supérieur au maximum.")

    return [
        [random.randint(minimum, maximum) for _ in range(colonnes)]
        for _ in range(lignes)
    ]
```

Génère une table 2D remplie de valeurs entières aléatoires dans un intervalle défini.

| | |
|---|---|
| **Paramètres** | lignes (int), colonnes (int), minimum (int, défaut=0), maximum (int, défaut=100) |
| **Retour** | Table2D — tableau rempli aléatoirement |
| **Erreurs** | ValueError si lignes/colonnes <= 0 ou minimum > maximum |

tableau = generer_tableau_aleatoire(3, 4, 1, 20)

- ## dimensions_tableau(tableau)

```python
def dimensions_tableau(tableau: Table2D) -> Tuple[int, int]:
    """
    Vérifie que le tableau 2D est valide et retourne ses dimensions.

    Un tableau 2D valide doit :
    - contenir au moins une ligne
    - contenir au moins une colonne
    - avoir le même nombre de colonnes sur chaque ligne
    """
    if not tableau or not tableau[0]:
        raise ValueError("Le tableau 2D ne doit pas être vide.")

    nb_colonnes = len(tableau[0])
    for ligne in tableau:
        if len(ligne) != nb_colonnes:
            raise ValueError("Toutes les lignes doivent avoir le même nombre de colonnes.")

    return len(tableau), nb_colonnes
```

Valide la structure d'un tableau 2D (non vide, lignes de même longueur) et retourne ses dimensions.

| | |
|---|---|
| **Retour** | Tuple[int, int] — (nombre_lignes, nombre_colonnes) |
| **Erreurs** | ValueError si tableau vide ou lignes de tailles différentes |

- ## aplatir_tableau(tableau)

```python
def aplatir_tableau(tableau: Table2D) -> List[int]:
    """
    Transforme une table 2D en liste 1D.

    Exemple :
    [[3, 1], [5, 2]] -> [3, 1, 5, 2]
    """
    dimensions_tableau(tableau)
    return [element for ligne in tableau for element in ligne]
```

Transforme une table 2D en une liste 1D en parcourant les lignes de gauche à droite.

[[3, 1], [5, 2]] → [3, 1, 5, 2]

- ### reconstruire_tableau(elements, lignes, colonnes)

```python
def reconstruire_tableau(elements: List[int], lignes: int, colonnes: int) -> Table2D:
    """
    Reconstruit une table 2D à partir d'une liste 1D en gardant les dimensions.

    Exemple :
    [1, 2, 3, 4], 2, 2 -> [[1, 2], [3, 4]]
    """
    if len(elements) != lignes * colonnes:
        raise ValueError("Le nombre d'éléments ne correspond pas aux dimensions demandées.")

    return [elements[i * colonnes:(i + 1) * colonnes] for i in range(lignes)]
```

Reconstruit une table 2D depuis une liste 1D, en découpant en tranches de taille 'colonnes'.

$[1, 2, 3, 4], 2, 2 \rightarrow [[1, 2], [3, 4]]$

- ### afficher_tableau(tableau)

```python
def afficher_tableau(tableau):
    max_val = max(max(ligne) for ligne in tableau)
    largeur = len(str(max_val)) + 2

    for ligne in tableau:
        print("|", end="")
        for val in ligne:
            print(f"{val:{largeur}}", end="")
        print(" |")
```

Affiche le tableau dans la console avec un formatage aligné. La largeur de chaque cellule s'adapte automatiquement à la valeur maximale présente dans le tableau.

## 2.4 Algorithmes de tri

Tous les algorithmes de tri suivent la même structure : aplatissement, tri de la liste 1D, reconstruction. Ils produisent tous un résultat trié en ordre croissant.

- ### Tri à bulle — tri_bulle(tableau)

Principe : comparaison successive de deux éléments voisins. Si l'élément de gauche est plus grand, on les échange. À chaque passage, le plus grand élément non trié remonte en fin de liste (comme une bulle).

Optimisation : si aucun échange n'est effectué lors d'un passage, la liste est déjà triée et l'algorithme s'arrête.

| | |
|---|---|
| **Complexité temporelle** | O(n²) dans le cas moyen et pire, O(n) dans le meilleur cas |
| **Complexité spatiale** | O(1) — tri en place sur la liste aplatie |
| **Stable** | Oui |

```python
def tri_bulle(tableau: Table2D) -> Table2D:

    Idée de l'algorithme :
    - On compare deux éléments voisins.
    - Si l'élément de gauche est plus grand que celui de droite, on les échange.
    - À chaque passage, les plus grands éléments "remontent" progressivement vers la fin.
    - On répète jusqu'à ce que toute la liste soit triée.

    Avec une table 2D :
    1) On l'aplatit en liste 1D.
    2) On applique le tri à bulle.
    3) On reconstruit la table 2D.
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)
    n = len(elements)

    for i in range(n):
        # À chaque tour, le plus grand élément restant se place en fin de liste.
        echange = False
        for j in range(0, n - i - 1):
            if elements[j] > elements[j + 1]:
                elements[j], elements[j + 1] = elements[j + 1], elements[j]
                echange = True
        # Si aucun échange n'a eu lieu, la liste est déjà triée.
        if not echange:
            break

    return reconstruire_tableau(elements, lignes, colonnes)
```

- ## Tri par sélection — tri_selection(tableau)

Principe : à chaque itération, on recherche le minimum dans la partie non triée et on le place à la position courante. On répète jusqu'à ce que toute la liste soit triée.

| | |
|---|---|
| **Complexité temporelle** | O(n²) dans tous les cas |
| **Complexité spatiale** | O(1) |
| **Stable** | Non |

```python
def tri_selection(tableau: Table2D) -> Table2D:
    """
    Trie une table 2D par TRI PAR SÉLECTION.

    Idée de l'algorithme :
    - On cherche le plus petit élément dans la partie non triée.
    - On le place au début.
    - On recommence pour la position suivante.

    Avec une table 2D :
    1) Aplatissement.
    2) Tri par sélection.
    3) Reconstruction.
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)
    n = len(elements)

    for i in range(n):
        indice_min = i
        for j in range(i + 1, n):
            if elements[j] < elements[indice_min]:
                indice_min = j
        elements[i], elements[indice_min] = elements[indice_min], elements[i]

    return reconstruire_tableau(elements, lignes, colonnes)
```

- ## Tri par insertion — tri_insertion(tableau)

Principe : on maintient une partie gauche triée. Pour chaque nouvel élément, on le déplace vers la gauche jusqu'à trouver sa position correcte, en décalant les éléments plus grands vers la droite.

| | |
|---|---|
| **Complexité temporelle** | O(n²) pire cas, O(n) meilleur cas (liste presque triée) |
| **Complexité spatiale** | O(1) |
| **Stable** | Oui |

```python
def tri_insertion(tableau: Table2D) -> Table2D:
    Trie une table 2D par TRI PAR INSERTION.

    Idée de l'algorithme :
    - On considère que la partie gauche est déjà triée.
    - On prend l'élément suivant.
    - On l'insère à la bonne place dans la partie déjà triée.

    Avec une table 2D :
    1) Aplatissement.
    2) Tri par insertion.
    3) Reconstruction.
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)

    for i in range(1, len(elements)):
        cle = elements[i]
        j = i - 1

        # On décale vers la droite tous les éléments plus grands que la clé.
        while j >= 0 and elements[j] > cle:
            elements[j + 1] = elements[j]
            j -= 1

        # On place ensuite la clé à la bonne position.
        elements[j + 1] = cle

    return reconstruire_tableau(elements, lignes, colonnes)
```

- ## Tri rapide — tri_rapide(tableau)

Principe : algorithme récursif de type « diviser pour régner ». On choisit un pivot (élément du milieu), on sépare les éléments en trois sous-listes (inférieurs, égaux, supérieurs), puis on trie récursivement chaque sous-liste.

Implémentation : la fonction interne _tri_rapide_liste() réalise le tri sur la liste 1D aplatie.

| | |
|---|---|
| **Complexité temporelle** | O(n log n) en moyenne, O(n²) en pire cas |
| **Complexité spatiale** | O(log n) en pile récursive |
| **Stable** | Non (dans cette implémentation) |

```python
def _tri_rapide_liste(elements: List[int]) -> List[int]:
    """Fonction interne : tri rapide sur une liste 1D."""
    # Cas de base : une liste vide ou à un seul élément est déjà triée.
    if len(elements) <= 1:
        return elements[:]

    # On choisit un pivot. Ici, on prend l'élément du milieu.
    pivot = elements[len(elements) // 2]

    # On découpe la liste en trois parties.
    inferieurs = [x for x in elements if x < pivot]
    egaux = [x for x in elements if x == pivot]
    superieurs = [x for x in elements if x > pivot]

    # On trie récursivement les sous-listes puis on les rassemble.
    return _tri_rapide_liste(inferieurs) + egaux + _tri_rapide_liste(superieurs)
```

```python
def tri_rapide(tableau: Table2D) -> Table2D:
    """
    Trie une table 2D par TRI RAPIDE.

    Idée de l'algorithme :
    - On choisit un pivot.
    - On sépare les éléments en trois groupes :
      * plus petits que le pivot
      * égaux au pivot
      * plus grands que le pivot
    - On applique récursivement la même méthode aux sous-groupes.

    Avec une table 2D :
    1) Aplatissement.
    2) Tri rapide.
    3) Reconstruction.
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)
    elements_tries = _tri_rapide_liste(elements)
    return reconstruire_tableau(elements_tries, lignes, colonnes)
```

- ## Tri par fusion — tri_fusion(tableau)

Principe : algorithme récursif de type « diviser pour régner ». On divise la liste en deux moitiés, on les trie récursivement, puis on les fusionne en une seule liste triée via la fonction _fusion().

La fonction _fusion() compare les éléments tête à tête depuis les deux sous-listes et construit progressivement la liste résultat.

| | |
|---|---|
| **Complexité temporelle** | O(n log n) dans tous les cas |
| **Complexité spatiale** | O(n) — liste intermédiaire nécessaire |
| **Stable** | Oui |

```python
def _fusion(gauche: List[int], droite: List[int]) -> List[int]:
    """Fusionne deux listes triées en une seule liste triée."""
    resultat = []
    i = 0
    j = 0

    # On compare l'élément courant de chaque liste et on ajoute le plus petit.
    while i < len(gauche) and j < len(droite):
        if gauche[i] <= droite[j]:
            resultat.append(gauche[i])
            i += 1
        else:
            resultat.append(droite[j])
            j += 1

    # On ajoute les éléments restants.
    resultat.extend(gauche[i:])
    resultat.extend(droite[j:])
    return resultat

def _tri_fusion_liste(elements: List[int]) -> List[int]:
    """Fonction interne : tri fusion sur une liste 1D."""
    if len(elements) <= 1:
        return elements[:]

    milieu = len(elements) // 2
    gauche = _tri_fusion_liste(elements[:milieu])
    droite = _tri_fusion_liste(elements[milieu:])
    return _fusion(gauche, droite)
```

```python
def tri_fusion(tableau: Table2D) -> Table2D:
    """
    Trie une table 2D par TRI PAR FUSION.

    Idée de l'algorithme :
    - On coupe la liste en deux parties.
    - On trie chaque moitié récursivement.
    - On fusionne ensuite les deux moitiés triées.

    Avec une table 2D :
    1) Aplatissement.
    2) Tri fusion.
    3) Reconstruction.
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)
    elements_tries = _tri_fusion_liste(elements)
    return reconstruire_tableau(elements_tries, lignes, colonnes)
```

- ### Tableau comparatif des algorithmes de tri

| Algorithme | Meilleur | Moyen | Pire | Espace | Stable |
|---|---|---|---|---|---|
| Tri à bulle | O(n) | O(n²) | O(n²) | O(1) | Oui |
| Tri sélection | O(n²) | O(n²) | O(n²) | O(1) | Non |
| Tri insertion | O(n) | O(n²) | O(n²) | O(1) | Oui |
| Tri rapide | O(n log n) | O(n log n) | O(n²) | O(log n) | Non |
| Tri fusion | O(n log n) | O(n log n) | O(n log n) | O(n) | Oui |

## 2.5 Algorithmes de recherche

Les fonctions de recherche retournent un Tuple (ligne, colonne) si la valeur est trouvée, ou None sinon. La position est calculée à partir de l'indice dans la liste aplatie par les formules : ligne = indice // colonnes et colonne = indice % colonnes.

- ## Recherche linéaire — recherche_lineaire(tableau, valeur)

Principe : parcours séquentiel de toutes les cellules du tableau (ligne par ligne, colonne par colonne). Dès que la valeur est trouvée, sa position est retournée.

Avantage majeur : fonctionne sur n'importe quel tableau, qu'il soit trié ou non.

| | |
|---|---|
| **Complexité** | O(n) — n étant le nombre total de cellules |
| **Précondition** | Aucune — fonctionne sur tout tableau |

```python
def recherche_lineaire(tableau: Table2D, valeur: int) -> Optional[Coordonnee]:
    """
    Recherche une valeur dans une table 2D par RECHERCHE LINÉAIRE.

    Idée de l'algorithme :
    - On parcourt chaque ligne.
    - Dans chaque ligne, on parcourt chaque colonne.
    - Dès qu'on trouve la valeur, on retourne sa position (ligne, colonne).

    Cette méthode fonctionne même si le tableau n'est pas trié.

    Retour :
    - (indice_ligne, indice_colonne) si trouvé
    - None sinon
    """
    dimensions_tableau(tableau)

    for i, ligne in enumerate(tableau):
        for j, element in enumerate(ligne):
            if element == valeur:
                return (i, j)
    return None
```

- ## Recherche binaire itérative — recherche_binaire(tableau, valeur)

Principe : on travaille sur la liste aplatie triée. On compare la valeur avec l'élément du milieu. Si inférieure, on cherche dans la moitié gauche ; si supérieure, dans la moitié droite. On répète avec une boucle while jusqu'à trouver ou épuiser la plage de recherche.

| | |
|---|---|
| **Complexité** | O(log n) |
| **Précondition** | Le tableau DOIT être trié en ordre croissant |

```python
def recherche_binaire(tableau: Table2D, valeur: int) -> Optional[Coordonnee]:
    Recherche une valeur dans une table 2D par RECHERCHE BINAIRE Iterative.

    Condition importante :
    - Le tableau doit déjà être trié en ordre croissant.

    Idée de l'algorithme :
    - On travaille sur la version aplatie du tableau.
    - On compare la valeur recherchée avec l'élément du milieu.
    - Si la valeur est plus petite, on cherche à gauche.
    - Si elle est plus grande, on cherche à droite.
    - On répète jusqu'à trouver l'élément ou épuiser la recherche.
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)

    gauche = 0
    droite = len(elements) - 1

    while gauche <= droite:
        milieu = (gauche + droite) // 2

        if elements[milieu] == valeur:
            return (milieu // colonnes, milieu % colonnes)
        if elements[milieu] < valeur:
            gauche = milieu + 1
        else:
            droite = milieu - 1

    return None
```

- ### Recherche par dichotomie récursive — recherche_dichotomie(tableau, valeur)

Principe identique à la recherche binaire, mais implémentée de manière récursive via la fonction interne _dichotomie_recursive(). La récursion s'arrête quand l'élément est trouvé ou quand la plage gauche > droite.

Différence par rapport à recherche_binaire : version récursive vs. version itérative. Le résultat est identique.

| | |
|---|---|
| **Complexité** | O(log n) |
| **Précondition** | Le tableau DOIT être trié en ordre croissant |
| **Pile récursive** | O(log n) niveaux |

```python
def _dichotomie_recursive(
    elements: List[int],
    valeur: int,
    gauche: int,
    droite: int,
) -> int:
    """
    Fonction interne : recherche par dichotomie récursive sur une liste triée.

    Retourne l'indice trouvé ou -1.
    """
    if gauche > droite:
        return -1

    milieu = (gauche + droite) // 2

    if elements[milieu] == valeur:
        return milieu
    if valeur < elements[milieu]:
        return _dichotomie_recursive(elements, valeur, gauche, milieu - 1)
    return _dichotomie_recursive(elements, valeur, milieu + 1, droite)
```

```python
def recherche_dichotomie(tableau: Table2D, valeur: int) -> Optional[Coordonnee]:
    """
    Recherche une valeur dans une table 2D par DICHOTOMIE récursive.

    Condition importante :
    - Le tableau doit déjà être trié en ordre croissant.

    Différence retenue ici :
    - recherche_binaire : version itérative
    - recherche_dichotomie : version récursive

    Retour :
    - (ligne, colonne) si trouvé
    - None sinon
    """
    lignes, colonnes = dimensions_tableau(tableau)
    elements = aplatir_tableau(tableau)
    indice = _dichotomie_recursive(elements, valeur, 0, len(elements) - 1)

    if indice == -1:
        return None

    return (indice // colonnes, indice % colonnes)
```

## III. Fichier exemple_utilisation.py

Ce fichier constitue une interface interactive en ligne de commande permettant à l'utilisateur de tester le module table2d_module. Il suit un flux séquentiel clair :

| Étape | Action | Détail |
|---|---|---|
| 1 | Génération du tableau | Tableau 6×6 avec valeurs entre 1 et 100 |
| 2 | Affichage initial | Tableau aléatoire affiché dans la console |
| 3 | Choix du tri | Menu de 5 options (1=Bulle, 2=Rapide, 3=Sélection, 4=Insertion, 5=Fusion) |
| 4 | Affichage du tableau trié | Résultat du tri choisi affiché |
| 5 | Saisie de la valeur | L'utilisateur entre la valeur à rechercher |
| 6 | Choix de la recherche | Menu de 3 options (1=Linéaire, 2=Binaire, 3=Dichotomie) |
| 7 | Affichage du résultat | Position (ligne, colonne) ou message 'non trouvée' |

Remarque importante : les recherches binaire et par dichotomie sont appliquées sur le tableau trié (tableau_trie), tandis que la recherche linéaire est appliquée sur le tableau d'origine (tableau) — ce qui illustre correctement l'absence de précondition de la recherche linéaire.

En cas de choix invalide pour le tri, le tri fusion est utilisé par défaut. En cas de choix invalide pour la recherche, le programme affiche un message d'erreur sans planter.

- EXEMPLE D'UTILISATION

```
=== TABLEAU INITIAL ===
|   50   24    7   79   93   39 |
|   26    1   66   18   45   82 |
|   52   13   96   48    2   13 |
|   89   17   22   32   13   26 |
|   88  100   89   42   14   64 |
|   66   79   14   12   27    2 |
```

```
=== CHOIX DU TRI ===
1 - Tri à bulle
2 - Tri rapide
3 - Tri par sélection
4 - Tri par insertion
5 - Tri par fusion
Choisissez un type de tri : 1

=== TABLEAU TRIÉ ===
|    1    2    2    7   12   13 |
|   13   13   14   14   17   18 |
|   22   24   26   26   27   32 |
|   39   42   45   48   50   52 |
|   64   66   66   79   79   82 |
|   88   89   89   93   96  100 |

=== RECHERCHE ===
Entrez la valeur à rechercher : 14

=== TYPE DE RECHERCHE ===
1 - Recherche linéaire
2 - Recherche binaire
3 - Recherche par dichotomie
Choisissez une méthode : 1
La valeur recherchée a plusieurs occurrences :
→ Position (Ligne, Colonne) : (1, 2)
→ Position (Ligne, Colonne) : (1, 3)
```

## IV. Gestion des erreurs et robustesse

Le module intègre une gestion rigoureuse des cas d'erreur :

| Condition d'erreur | Exception levée | Fonction concernée |
|---|---|---|
| lignes <= 0 ou colonnes <= 0 | ValueError | generer_tableau_aleatoire |
| minimum > maximum | ValueError | generer_tableau_aleatoire |
| Tableau vide ([] ou [[]]) | ValueError | dimensions_tableau |
| Lignes de tailles différentes | ValueError | dimensions_tableau |
| Mauvais nombre d'éléments pour reconstruction | ValueError | reconstruire_tableau |

Toutes les fonctions de tri et de recherche appellent dimensions_tableau() en début d'exécution, ce qui garantit une validation systématique de l'entrée.

## Conclusion

Ce projet illustre de manière pédagogique les principaux algorithmes de tri et de recherche appliqués à des structures de données 2D. L'approche adoptée — aplatissement en liste 1D, traitement, reconstruction — permet de réutiliser des algorithmes classiques 1D sans les modifier, tout en travaillant sur des tableaux bidimensionnels.

Le module présente plusieurs qualités : une documentation complète (docstrings en français), une gestion des erreurs robuste, des types annotés pour la lisibilité, et une séparation claire entre les fonctions utilitaires, de tri et de recherche.

Le fichier exemple_utilisation.py démontre efficacement les capacités du module à travers une interface interactive, couvrant l'ensemble des algorithmes disponibles.